

ROB550 Spring 2026 ArmLab Report

Michael Gareau, Behrad Rabiei, Myles Mackie
 gareaum@umich.edu , brabiei@umich.edu, mamackie@umich.edu

Abstract—This report details the development of computer vision algorithms and kinematic modeling to autonomously control a 5-DOF robotic arm. An overhead Intel RealSense LiDAR camera is utilized to identify the positions of blocks with varying colors and shapes. Camera calibration is performed using fiducial AprilTags with known positions to accurately compute the extrinsic matrix and apply homogeneous transformations to convert coordinates between world, camera, and pixel reference frames. To enable precise end-effector manipulation for grasping and placement tasks, forward and inverse kinematics are implemented to determine feasible joint configurations for desired poses. Overall, this development resulted in a 5-DOF robotic system embedded with perception and motion planning algorithms capable of reliably and autonomously detecting, grasping, and arranging blocks in a predetermined configuration.

I. INTRODUCTION

A robotic arm is a machine designed to move and function in ways similar to human arms. The primary components of robotic arms are the links and the end-effector. The links consist of rigid segments connected by joints that allow the arm to bend, rotate, or extend. The joints are powered by motors that enable controlled rotational motion or translational displacement. At the end of the arm is a tool known as the end-effector, which allows the robot to interact with its environment and perform specific tasks.

Robotic systems are embedded with control algorithms enabling them to operate within a defined workspace. These systems often incorporate sensors and cameras to collect information about their surroundings, enabling informed decisions regarding trajectory planning and motion control to safely and efficiently carry out tasks.

The development and control of robotic arms is a major area within robotics due to their versatility across a wide range of applications. Advancements in this field have enabled robotic systems to perform tasks that require high precision, repeatability, or operation in environments that may be hazardous or inaccessible to humans.

In this paper, the kinematic modeling and algorithms are developed to be integrated with the ReactorX 200 (RX200) robotic arm, a 5-DOF manipulator equipped with 7 Dynamixel servo motors and a 3D printed custom end effector. An Intel RealSense LiDAR

Camera and L515 sensor were used for computer vision and perception task. The camera contains both an RGB camera and solid state LiDAR depth sensor, which, when integrated with the ROS Realsense driver, allow it to interface with the sensor and capture the images needed for our project. When embedded with perception and motion planning algorithms, the robotic arm is able perform the following tasks: first, detect and accurately distinguish blocks based on varying color and sizes within the workspace; and second, autonomously grasp and arrange in a predefined configuration.

The remainder of this report is organized as follows. Section II describes the methodology used to develop the perception, kinematics, and motion planning algorithms for the RX200 robotic arm, including camera calibration, object detection, forward and inverse kinematics, and path planning strategies. Section III presents the results of integrating these algorithms with the robotic system, demonstrating block detection, grasping, and autonomous arrangement. Section IV provides a discussion of the system performance, limitations, and potential improvements. Finally, Section V concludes the report by summarizing and suggested improvements to enhance the autonomy and reliability of the robotic manipulation system.

II. METHODOLOGY

A. Workspace Transformation

To allow the robotic arm to interact with objects detected by the camera, points must be converted between pixel, camera, and world coordinate frames. This transformation is performed using the camera intrinsic matrix $\mathbf{K}$ and the extrinsic transformation matrix $\mathbf{H}$.

The intrinsic matrix maps normalized camera coordinates to image pixel coordinates. It can be obtained either through factory calibration or through a standard calibration procedure that minimizes reprojection error using multiple images of a calibration pattern.

The extrinsic matrix describes the rigid transformation between the world frame and the camera frame. Instead of relying on manual measurements, which are prone to error, the transformation is computed using fiducial AprilTags. Using the Robot Operating System (ROS), the camera detects the four AprilTags placed on the workspace and estimates their pose relative to the

camera. Given the known positions of these tags in the world frame, the Perspective-n-Point (PnP) algorithm is used to estimate the rotation and translation that define the extrinsic transformation.

Once the intrinsic and extrinsic parameters are known, pixel coordinates can be converted into 3D world coordinates. First, a pixel coordinate (u, v) with depth z is projected into the camera coordinate frame:

$$\mathbf{p}_C = z \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix} \quad (1)$$

where $\mathbf{p}_C$ represents the 3D point in the camera frame.

Next, the point is transformed from the camera frame to the world frame using the extrinsic transformation matrix:

$$\begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix} = \mathbf{H}^{-1} \begin{bmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{bmatrix} \quad (2)$$

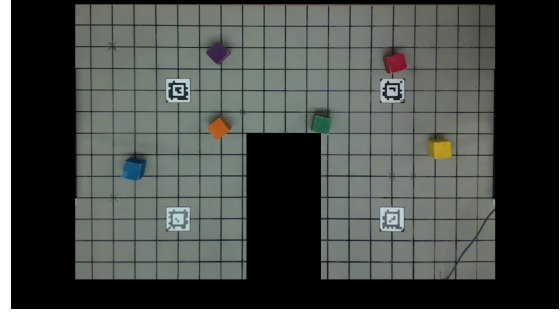
This sequence of transformations converts pixel detections from the camera into accurate 3D world coordinates, providing the robotic arm with precise spatial information about objects in its workspace. Conversely, applying these equations in reverse allows world coordinates to be mapped back to pixel coordinates, enabling accurate visualization and interaction within the image frame.

B. Block Detection

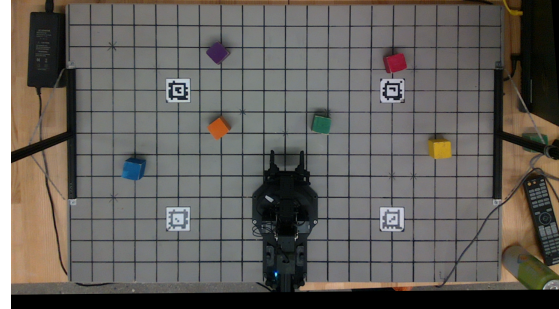
The block detection algorithm relies entirely on the RGB camera. The depth calibration we performed introduced a Z offset, making the world Z coordinate under the homography unreliable for segmenting blocks. Also, because our workstation was positioned near a west-facing window, the lighting conditions for the camera heavily depended on the time of day, making this an important environmental factor to account for. The block detection process consists of two main steps: light normalization and hue segmentation. Light normalization is a preprocessing step that adjusts the hue and brightness of images to account for lighting warmth and intensity across images, and hue segmentation is the process of identifying block locations based on their color.

1) *Light Normalization*: First, we use a static mask of the board to isolate where the camera looks for blocks, shown in Figure 1a. We treat the board as a neutral gray and use its masked image as a real-time reference for object color. Next, the image is “white balanced” by rescaling the RGB channels to match the

neutral grey reference mean. This balancing process accounts for any warmth or slight color changes in lighting that may impact the overall hue of the image, but it does not account for sharp shadows. This process assumes that the board occupies most of the pixels in its cropped image, that the board is a neutral grey, and that the total mean RGB values of the entire image are reasonably close to this grey. We take the mean of the RGB channels separately and use their ratios to the mean grey as scale factors for each channel. This operation adjusts the RGB channels separately to the mean grey value, “balancing” the image’s hue shown in Figure 1b.



(a) RGB Image with Board Mask



(b) White Balanced Image

Fig. 1: Image White Balancing

After white balancing, we boost the image’s brightness to exaggerate the difference between the saturated block colors and the unsaturated grey board. To do this, we rescale each RGB channel from its neutral grey mean to pure white, then clip values above 255. This results in a “blown out” image with distinctly saturated regions representing blocks shown in Figure 2a. We convert the image to HSV and apply masks to eliminate insufficiently saturated or bright regions as displayed in Figure 2b.

2) *Hue Segmentation*: Segmenting the blocks requires applying hue masks for each color and drawing contours around what remains in the image. The hue ranges were tuned manually to properly distinguish between red, orange, yellow, green, blue, and purple. Contours are filtered by size to ignore any small noisy

artifacts. The remaining contours' center locations represent the blocks' pixel locations in the image, shown in Figure 2c.

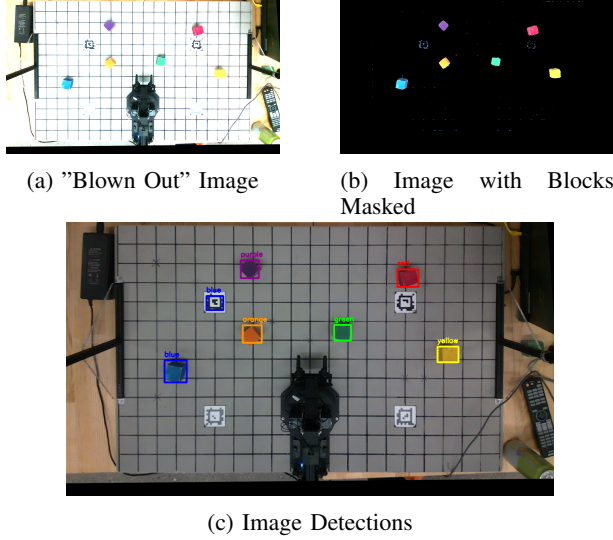


Fig. 2: Hue Segmentation

C. Homography

The camera used in this system is mounted above the workspace with a slight positional and angular offset, causing the captured image of the board to appear skewed rather than perfectly aligned with the image frame. To correct this distortion and represent the workspace as if viewed from directly above, a projective transformation was applied using a homography matrix.

To compute the homography matrix, reference points were obtained using the detected positions of AprilTags within the workspace. The pixel coordinates of the AprilTags provided by the detection algorithm served as source points. The corresponding destination points were defined to represent the desired locations of these features. These destination points were arranged such that the workspace would appear as a properly scaled rectangle in the transformed image. Using these point correspondences, a homography matrix was calculated to map the original camera image to the rectified coordinate frame. This projective transformation corrects the perspective distortion caused by the camera's angled viewpoint. As a result, the workspace can be treated as if it were viewed directly from above, which simplifies subsequent perception tasks such as object localization and coordinate estimation.

Following the homography transformation, the rectified image was integrated into the graphical user interface to allow world coordinates to be computed from pixel locations within the workspace. When the user moves the mouse across the image, the corresponding

pixel location is converted back to world coordinates using the calibration parameters and depth information. To verify the accuracy of the calibration, known reference points on the board were measured and compared with their computed world coordinates

D. Forward Kinematics

Forward kinematics is a method to determine the position of the end effector ($\mathbf{X}$), where $\mathbf{X} = [xyz\theta\psi\phi]$ in the workspace given joint angle $\mathbf{q}$, where $\mathbf{q} = [\theta_1\theta_2\theta_3\theta_4\theta_5]$ in the configuration space. Each θ corresponds to one joint of the 5-DOF robotic arm. To solve for the forward kinematics, the Denavit-Hartenberg(DH) method is utilized.

The DH method describes the spatial relationship between consecutive links by assigning coordinate frames to each joint and expressing the transformation between them using four parameters: joint angle(θ_i), link offset(d_i), link length(a_i), and link twist(α_i).

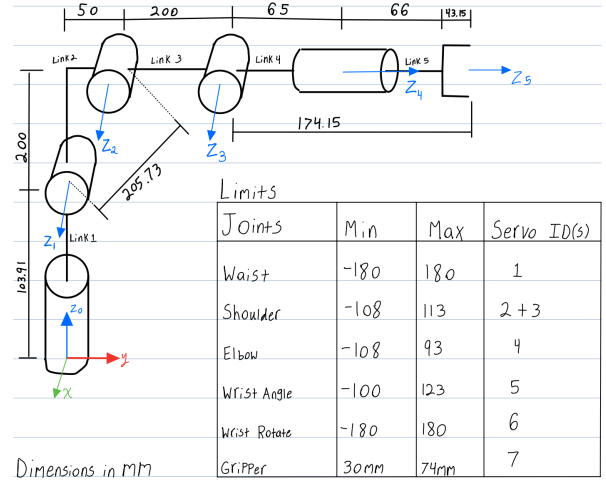


Fig. 3: Schematics of the robot arm used to calculate forward kinematics using DH

Link	θ_i	d_i	a_i	α_i
1	$\theta_1^* + 90$	103.91	0	$-\pi/2$
2	$\theta_2^* - 75.96$	0	205.73	0
3	$\theta_3^* + 75.96$	0	200	0
4	$\theta_4^* - 90$	0	0	$-\pi/2$
5	θ_5^*	152.575	0	0

TABLE I: DH parameters

To define these parameters, the links and joints of the arm must first be identified. Once all links and joints are determined, the z-axis for each joint can be

defined along the axis of rotation. The base frame is then selected, and the x -axes are determined using the rule that axis x_i must be perpendicular to axis z_{i-1} and intersect it. From there, by working through each joint, the required parameters can be identified. Each row of the DH table represents a joint and can be used to construct a homogeneous transformation matrix which is constructed as follows:

$$\begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The complete forward kinematics equation is assembled from the individual homogeneous transformations, resulting in the final homogeneous transformation matrix H

$$H = A_1(q_1) \dots A_n(q_n) = \begin{bmatrix} R_n^0 & O_n^0 \\ 0 & 1 \end{bmatrix} \quad (3)$$

Here R_n^0 represents the rotation matrix of the end effector and O_n^0 represents the position of the end effector with respect to the world frame. Additionally, the Euler angles θ , ψ , and ϕ describing the orientation of the end effector can be computed from the rotation matrix.

E. Inverse Kinematics

The inverse kinematics (IK) problem determines the joint configuration $\Theta = [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5]^T$ required to reach a desired end-effector pose $\mathbf{p}_d = [x_d, y_d, z_d, \phi]^T$. For the RX200 manipulator, the kinematic structure allows the problem to be decomposed into an azimuthal rotation about the base and a planar positioning problem for the remaining joints.

1) *Base Alignment*: The first joint rotates the arm about the vertical axis to align the manipulator with the target in the horizontal plane. This rotation is computed analytically using the desired Cartesian coordinates

$$\theta_1 = -\text{atan2}(x_d, y_d). \quad (4)$$

The wrist roll θ_5 is then chosen to maintain the desired tool orientation relative to the base

$$\theta_5 = \theta_1 + \gamma. \quad (5)$$

After this step, the remaining problem reduces to positioning the arm within a vertical plane defined by the target point.

2) *Planar Positioning*: Let $r_d = \sqrt{x_d^2 + y_d^2}$ denote the radial distance from the base axis. The shoulder,

elbow, and wrist pitch joints $\theta_p = [\theta_2, \theta_3, \theta_4]^T$ determine the end-effector position in this plane. Using the link lengths L_0, L_1, L_2 , and L_3 , the forward kinematics constraints become

$$r_d = L_1 \cos(\theta'_2) + L_2 \cos(\theta_{23}) + L_3 \cos(\theta_{234}) \quad (6)$$

$$z_d = L_0 + L_1 \sin(\theta'_2) + L_2 \sin(\theta_{23}) + L_3 \sin(\theta_{234}) \quad (7)$$

where

$$\theta_{23} = \theta_2 + \theta_3, \quad \theta_{234} = \theta_2 + \theta_3 + \theta_4,$$

and $\theta'_2 = -\theta_2 + \alpha$ accounts for the fixed shoulder offset α .

The desired wrist pitch imposes the additional constraint

$$\theta_2 + \theta_3 + \theta_4 = \psi. \quad (8)$$

These equations define a nonlinear system that is solved using a Newton-Raphson iterative method. The solver updates the joint vector θ_p until the residual between the forward kinematics and the desired pose is minimized.

3) *Feasible Solution Selection*: To ensure physically valid configurations, the implementation checks that each joint remains within its mechanical limits. If a candidate solution violates the bounds (e.g., $|\theta_i| > \pi$), the target pose is slightly perturbed and the solver is restarted. This prevents the algorithm from converging to singular or unreachable configurations.

4) *Click-to-Grab Motion*: Object interaction is executed using a three-stage waypoint trajectory to improve grasp stability:

- **Approach**: $\mathbf{p}_{hover} = [x_d, y_d, z_d + \delta]^T$
- **Grasp**: $\mathbf{p}_{grab} = [x_d, y_d, z_d]^T$
- **Retract**: $\mathbf{p}_{clear} = [x_d, y_d, z_d + \delta]^T$

Approaching from above reduces steady-state error caused by gravity and improves grasp reliability.

III. RESULTS

A. Calibration

The following matrices were obtained through several calibration methods as shown in tables 2-6.

918.360	0.0	661.192
0.0	919.153	356.597
0.0	0.0	1.0

TABLE II: Intrinsic Matrix(Factory Calibrated)

962.028	0.0	675.372
0.0	958.618	329.020
0.0	0.0	1.0

TABLE III: Intrinsic matrix(Checkerboard Calibrated)

0.999	$1.787e^{-3}$	$-1.262e^{-2}$	19.973
$3.269e^{-3}$	-0.993	0.118	215.714
$1.233e^{-2}$	-0.118	-0.993	$1.026e^3$
0.0	0.0	0.0	1.0

TABLE IV: Extrinsic Matrix(Apriltag Calibration)

1	0	0	0
0	-0.981	0.191	219
0	0.191	-0.981	1016
0.0	0.0	0.0	1.0

TABLE V: Extrinsic Matrix(Hand Measurement)

1.047	$-7.212e^{-2}$	$-6.962e^1$
$-2.590e^{-3}$	1.018	$-1.027e^2$
$8.968e^{-6}$	$-1.155e^{-4}$	1.0

TABLE VI: Homography Matrix

Overall, the most accurate results were obtained using the factory-calibrated intrinsic matrix combined with the extrinsic matrix computed through AprilTag calibration. These also intuitively seemed more accurate as there is no source of human error compared to the checkerboard and hand measurement method.

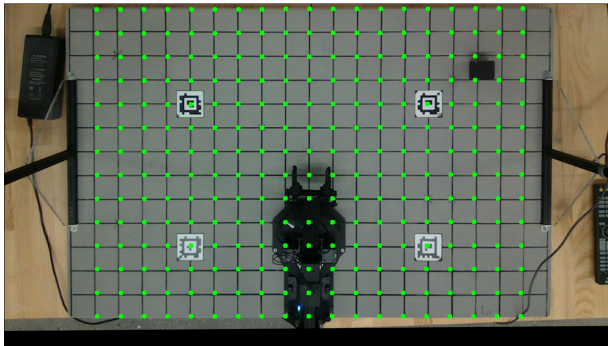


Fig. 4: Grid point project with homography applied

B. Block Detection

We divided the board into 5 regions shown in Figure 5 to consider how the detection system performs

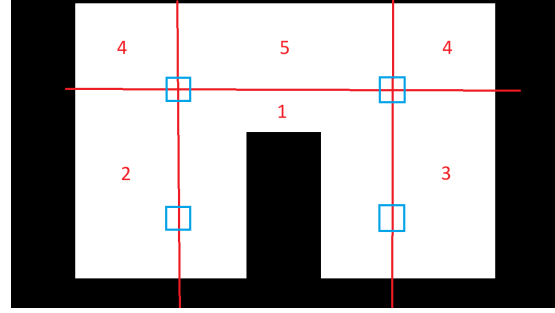


Fig. 5: Board Mask Divided into Sections

for a large number of blocks in different positions on the board. Twelve blocks (red, orange, yellow, green, blue, and purple, two each) were placed in each region to determine where and how the detection system might fail in a worse case scenario. We found that the detection system performs the most consistently when the camera image see the blocks directly face down without reflections, which occurs in regions 1 and 5. Of the blocks placed in these regions, red, green, and yellow were perceived best. Purple blocks were most consistently either mislabeled as blue due to their similar hue or unlabeled altogether due to its lack of saturation against the board. In regions 2, 3 and 4, the system tends to double label some blocks, perceiving its top face as one color and its visible side face as another color. This happened consistently with orange and yellow blocks (being dual labeled with red and orange respectfully) and green and purple blocks (both gaining a secondary blue label). When considering one block at a time, the system performs quite well. Purple and blue blocks are still consistently the worst performing colors, but the closer they are to the middle of the board, the better the system performs with them. Regularly one or two of the AprilTags are mislabeled as a blue block, which didn't degrade performance in our use cases, but it is something to note.

C. Cycle Task

To learn how to make the robot move to certain joint positions a simple teach and repeat task was conducted. The task consisted of teaching the robot to cycle swapping blocks at certain locations through an intermediate location. During the task, the robot was able to achieve 7 cycles. Fig. 6 shows the plot of the joint angles over time for 1 cycle.

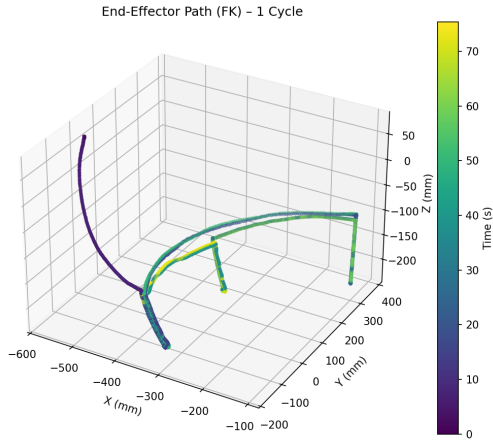


Fig. 6: Block cycling end-effector trajectory

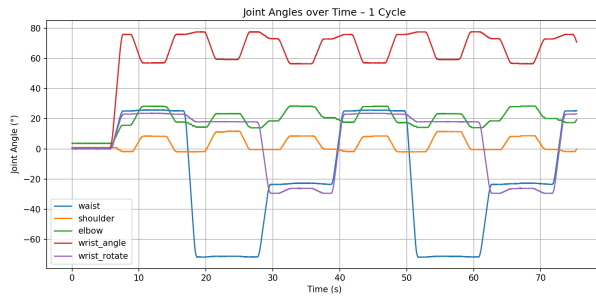


Fig. 7: Plot of Block cycling joint angles for 1 cycle

D. Forward Kinematics

To evaluate the accuracy of the forward kinematics model, the end effector was manually positioned at several locations across the robot's workspace. The desired positions were compared with the measured positions obtained from the robotic system. The results of these trials are summarized in Table VII.

As shown in Table VII, the positional error ranged from 4.5 to 7.5, with an overall average error 6.25mm. This level of error is relatively small compared to the size of the workspace and is considered acceptable for the block manipulation tasks performed in this project. Therefore, the forward kinematics model provides sufficient positional accuracy for reliable grasping and placement operations.

$x_{desired}$	$y_{desired}$	x_{actual}	y_{actual}	Pos. error (avg.)
-400	-100	-398	-107	4.5
-250	-150	-254	-141	7.5
-250	150	-243	144	6.5
250	150	245	141	7
250	-150	243	-144	6.5
400	-100	393	-104	5.5

TABLE VII: Forward kinematics results (mm)

IV. DISCUSSION

A. Calibration

The workspace calibration process produced reasonably accurate results for mapping image coordinates to physical positions on the board. Using the factory-calibrated intrinsic matrix together with the extrinsic matrix obtained from AprilTag calibration minimized error and improved the reliability of converting between pixel, camera, and world coordinate frames. The homography transformation further improved usability by correcting the perspective distortion caused by the angled camera placement. After applying this correction, the workspace appeared rectified, allowing points on the board to be interpreted as if the camera were positioned directly overhead.

However, the values along the z -axis were less consistent than those in the x and y directions. As the mouse moved farther from the center of the board, the computed z value gradually changed, reaching approximately 10 mm near the borders in the y direction. One possible source of this error is noise in the AprilTag detection process, which can introduce small inaccuracies in the estimated camera pose and therefore affect the extrinsic matrix. Another potential source is noise in the depth sensor used to estimate the distance from the camera to the workspace. While this level of error was acceptable for shorter stacks, it becomes problematic as stack height increases. Increasing the number of AprilTags used during extrinsic calibration or averaging pose estimates across multiple frames could help reduce this error and improve overall calibration accuracy.

B. Color and Block Detection

The detection system performed reliably under varying uniform lighting conditions, especially for blocks near the center of the board. Of the objective colors (red, orange, yellow, green, blue, and purple), orange and blue were the most difficult to pin down. Orange blocks were the most sensitive to reflections and shadows, causing them to either be mislabeled as yellow or red, while blue blocks were the hardest to detect at times.

The system has issues with darker blue and purple blocks as a result of the hue segmentation method because the lack of saturation causes the value and saturation preprocessing steps to erase them from the block detection mask. To accommodate for this lack of saturation, we widened the detection system saturation bounds, resulting in the mislabeling one of the AprilTags as a blue block as well. We viewed this as a necessary compromise to ensure that the all blue blocks remain visible to the detection system.

Blocks that are shaded in between the primary objective colors can be inconsistent. Occasionally the magenta blocks will be labelled red or purple, and the red-orange blocks will be labelled either red or orange. For the lighter shaded blocks, there are some mislabeling issues that stem from reflections, causing for example light green blocks to appear blue, or orange blocks to appear yellow. Direct sunlight was a challenge we ultimately solved by blocking it out, opting to keep the board as evenly lit as possible.

Depth segmentation likely would have improved the system's ability to detect blocks by removing the misidentified AprilTag and allowing for orientation measurement. Block angles from the RGB measurement system proved to be inconsistent due to the noisy nature of masking by saturation and value.

C. Forward and Inverse Kinematics

The forward kinematics algorithm performed well overall, with the majority of the error observed in the y-direction. This error likely arises from two main sources. First, inaccuracies in the homography transformation can introduce small distortions when converting between camera and world coordinates, particularly along axes that are not perfectly aligned with the image plane. Second, the forward kinematics model relies on precise Denavit-Hartenberg parameters and assumptions about the rigidity and alignment of the robotic arm's links and joints. Minor discrepancies in joint placement, link lengths, or motor positioning can propagate through the kinematic chain, resulting in deviations at the end effector. Addressing these sources of error would further improve the accuracy of the end-effector position. Nonetheless, the observed deviations are relatively small, with an average positional error of approximately 6.25 mm, which is acceptable for the block manipulation tasks performed in this lab.

D. Geometric vs Numerical Inverse Kinematics

Inverse kinematics can generally be solved using either geometric (closed-form) methods or numerical (iterative) methods. Geometric approaches derive analytical expressions for the joint angles using the manipulator's

link lengths and joint relationships. These methods are computationally efficient and provide deterministic solutions, but they can become difficult to derive for manipulators with complex kinematic structures or orientation constraints. Numerical inverse kinematics instead treats the problem as a nonlinear system that is solved iteratively. Methods such as Newton-Raphson use forward kinematics and the local error between the current and desired pose to update the joint configuration. Numerical approaches are more general and easier to implement for complex robots, but they may depend on the initial guess and can encounter convergence issues near singularities.

In this work, a hybrid strategy combining geometric reasoning and numerical optimization was used. The base rotation was solved analytically to align the manipulator with the target in the horizontal plane, reducing the remaining problem to a planar positioning task. The shoulder, elbow, and wrist pitch angles were then computed using a Newton-Raphson solver to satisfy the nonlinear forward kinematics constraints.

This hybrid approach leverages the efficiency of geometric solutions where possible while maintaining the flexibility of numerical methods for solving the remaining nonlinear equations. As a result, the method provides a practical and robust inverse kinematics solution for the RX200 manipulator.

V. CONCLUSION

The ReactorX200 robotic arm was successfully integrated with computer vision, kinematic modeling, and motion planning algorithms, enabling autonomous block detection, grasping, and arrangement. Using computer vision algorithms, the camera could accurately and reliably determine color, size, and orientation of each block. Additionally, the motion planning and control algorithms were capable of grasping and arranging blocks with minimal errors; however, further tuning of these algorithms could improve stacking performance. Overall, the robot system was able to achieve success across majority of the defined metrics, demonstrating the effectiveness of the integrated perception and motion planning framework.

REFERENCES

- [1] M. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2005. [Online]. Available: <https://books.google.com/books?id=wGapQAACAAJ>